

Bloc 2 MVC PHP TD6

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	4
Exercice 4	6
Exercice 5	7
Exercice 6	9
Exercice 7	10
Exercice 8	12
Exercice 9	14
Exercice 10	15
Exercice 11	16
Exercice 12	17
Exercice 13	18
Exercice 14	21
Exercice 15	23
Exercice 16	25
Conclusion	31

Contexte

Ce **TD** est la suite du **TD5**. Le but est d’avoir une interface “administrateur” qui gère les voitures, les utilisateurs et les trajets (**CRUD** : Create / Read / Update / Delete), en rendant notre modèle et nos contrôleurs génériques pour ne pas tout recopier.

(Les fichiers projet seront disponibles dans le portfolio sur GitHub).

Exercice 1

Si aucune action n’est donnée dans l’URL, on initialise **\$action** à “readAll” dans le routeur avec **isset()** :

```
<?php
require_once File::build_path(array("controller", "ControllerVoiture.php"));

if (isset($_GET['action'])) {
    $action = $_GET['action'];
} else {
    $action = 'readAll';
}

// Appel de la méthode statique $action de ControllerVoiture
ControllerVoiture::$action();
?>
```

Test en appelant “**index.php**” sans action :

 http://localhost/public_html/PHP/TD6/index.php

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Liste des voitures :

Voiture d'immatriculation [&immat=h.](#)

Voiture d'immatriculation [00-000-00.](#)

Voiture d'immatriculation [<h1>Hack.](#)

Voiture d'immatriculation [AA-000-AA.](#)

Voiture d'immatriculation [AB-123-CD.](#)

Voiture d'immatriculation [CI-789-TR.](#)

Voiture d'immatriculation [FI-456-AT.](#)

Voiture d'immatriculation [XX-999-YY.](#)

[Ajouter une voiture](#)

Exercice 2

Le routeur vérifie que l'action est bien une méthode de **ControllerVoiture** avec `get_class_methods()` et `in_array()`, sinon il appelle l'action **error** :

```
if (in_array($action, get_class_methods('ControllerVoiture'))) {  
    ControllerVoiture::$action();  
} else {  
    ControllerVoiture::error();  
}  
?>
```

Création de l'action **error** dans le contrôleur, elle affiche la vue **“view/voiture/error.php”** :

```
public static function error() {  
    $controller = 'voiture';  
    $view = 'error';  
    $pagetitle = 'Erreur';  
    require File::build_path(array("view", "view.php"));  
}
```

Test avec une action qui n'existe pas :



A screenshot of a web browser window. The address bar shows the URL: `http://localhost/public_html/PHP/TD6/index.php?action=nimportequoi`. Below the address bar, there are three links: [Voitures](#), [Utilisateurs](#), and [Trajets](#). The main content area displays an error message: "Erreur : la page demandée n'existe pas ou aucune voiture ne correspond à cette immatriculation." Below the error message is a link: [Retour à la liste](#). At the bottom of the page, there is a horizontal line.

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Erreur : la page demandée n'existe pas ou aucune voiture ne correspond à cette immatriculation.

[Retour à la liste](#)

Exercice 3

Ajout de la fonction “**deleteByImmat(\$immat)**” dans le modèle, avec une requête préparée :

```
public static function deleteByImmat($immat) {
    $sql = "DELETE FROM voiture WHERE immatriculation=:nom_tag";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "nom_tag" => $immat,
    );
    $req_prep->execute($values);
}
```

Ajout de l’action **delete** dans le contrôleur, elle supprime la voiture puis initialise **\$immat** et **\$tab_v** avant d’appeler la vue générique :

```
public static function delete() {
    $immat = $_GET['immat'];
    ModelVoiture::deleteByImmat($immat);
    $tab_v = ModelVoiture::getAllVoitures();
    $controller = 'voiture';
    $view = 'deleted';
    $pagetitle = 'Voiture supprimée';
    require File::build_path(array("view", "view.php"));
}
```

Création de la vue “**deleted.php**” (message + liste des voitures comme “**created.php**”) :

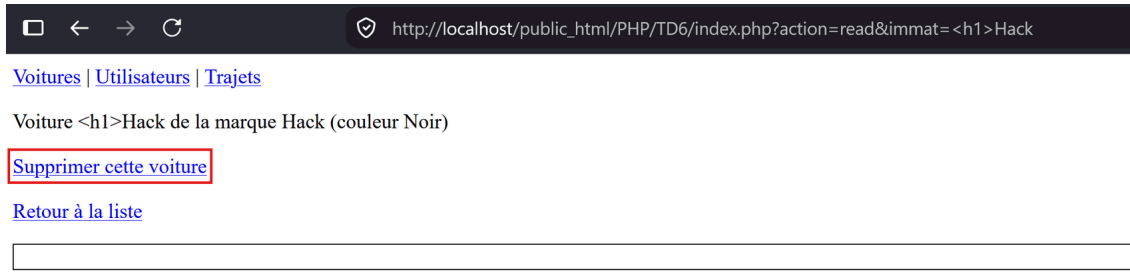
```
<p>La voiture d'immatriculation <?php echo htmlspecialchars($immat); ?> a bien été supprimée !</p>
<?php require File::build_path(array("view", "voiture", "list.php")); ?>
```

Ajout d’un lien pour supprimer la voiture dans la vue de détail :

```
<?php
$vImmatriculation = htmlspecialchars($v->getImmatriculation());
$vMarque = htmlspecialchars($v->getMarque());
$vCouleur = htmlspecialchars($v->getCouleur());
$vImmatriculationURL = rawurlencode($v->getImmatriculation());

echo "<p>Voiture " . $vImmatriculation . " de la marque " . $vMarque . " (couleur " . $vCouleur . ")</p>";
?>
<p><a href="index.php?action=delete&immat=<?php echo $vImmatriculationURL; ?>">Supprimer cette voiture</a></p>
<a href="index.php?action=readAll">Retour à la liste</a>
```

On va supprimer la voiture “<h1>Hack” créée au TD5 :



The screenshot shows a web browser with the address bar displaying `http://localhost/public_html/PHP/TD6/index.php?action=read&immat=<h1>Hack`. Below the address bar, there are navigation links: [Voitures](#) | [Utilisateurs](#) | [Trajets](#). The main content area displays the text "Voiture <h1>Hack de la marque Hack (couleur Noir)". Below this text, there is a button labeled "Supprimer cette voiture" which is highlighted with a red rectangular border. At the bottom of the content area, there is a link "Retour à la liste".



The screenshot shows the same web browser after the delete action. The address bar now displays `http://localhost/public_html/PHP/TD6/index.php?action=delete&immat=<h1>Hack`. The navigation links remain the same. The main content area now displays the message "La voiture d'immatriculation <h1>Hack a bien été supprimée !". Below this message, there is a section header "Liste des voitures :". Under the header, there is a list of cars with their license plates: [Voiture d'immatriculation &immat=h.](#), [Voiture d'immatriculation 00-000-00.](#), [Voiture d'immatriculation AA-000-AA.](#), [Voiture d'immatriculation AB-123-CD.](#), [Voiture d'immatriculation CI-789-TR.](#), [Voiture d'immatriculation FI-456-AT.](#), and [Voiture d'immatriculation XX-999-YY.](#). At the bottom of the list, there is a link "Ajouter une voiture".

La voiture a bien été supprimée.

Exercice 4

Création de la vue “**update.php**” : même formulaire que “**create.php**” mais pré-rempli avec l’attribut **value**, l’immatriculation est en **readonly** et le champ caché envoie “**action=updated**” :

```
<?php
$vImmatriculation = htmlspecialchars($v->getImmatriculation());
$vMarque = htmlspecialchars($v->getMarque());
$vCouleur = htmlspecialchars($v->getCouleur());
?>
<form method="get" action="index.php">
  <fieldset>
    <legend>Modifier la voiture :</legend>
    <input type="hidden" name="action" value="updated" />
    <p>
      <label for="immat_id">Immatriculation</label> :
      <input type="text" name="immatriculation" id="immat_id" value="<?php echo $vImmatriculation; ?>" readonly/>
    </p>
    <p>
      <label for="marque_id">Marque</label> :
      <input type="text" placeholder="Ex : Opel" name="marque" id="marque_id" value="<?php echo $vMarque; ?>" required autofocus/>
    </p>
    <p>
      <label for="couleur_id">Couleur</label> :
      <input type="text" placeholder="Ex : Rouge" name="couleur" id="couleur_id" value="<?php echo $vCouleur; ?>" required/>
    </p>
    <p>
      <input type="submit" value="Envoyer" />
    </p>
  </fieldset>
</form>
```

Action **update** du contrôleur, on récupère la voiture avec “**getVoitureByImmat()**” :

```
public static function update() {
    $v = ModelVoiture::getVoitureByImmat($_GET['immat']);
    if ($v === false) {
        self::error();
    } else {
        $controller = 'voiture';
        $view = 'update';
        $pagetitle = 'Modifier une voiture';
        require File::build_path(array("view", "view.php"));
    }
}
```

Et un lien “Modifier cette voiture” dans “**detail.php**” :

```
<p><a href="index.php?action=update&immat=<?php echo $vImmatriculationURL; ?>">Modifier cette voiture</a></p>
```

Voitures | Utilisateurs | Trajets

Modifier la voiture :

Immatriculation : AB-123-CD

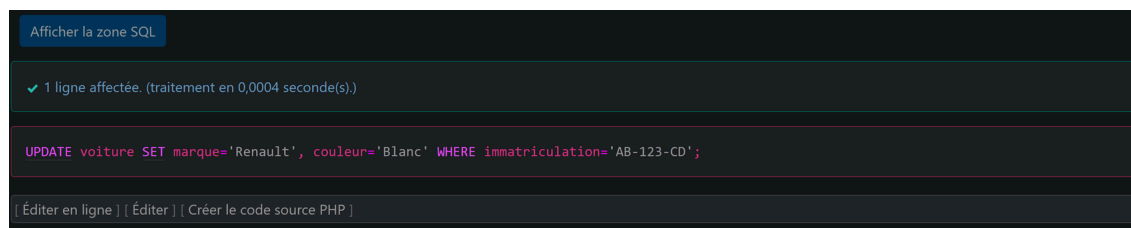
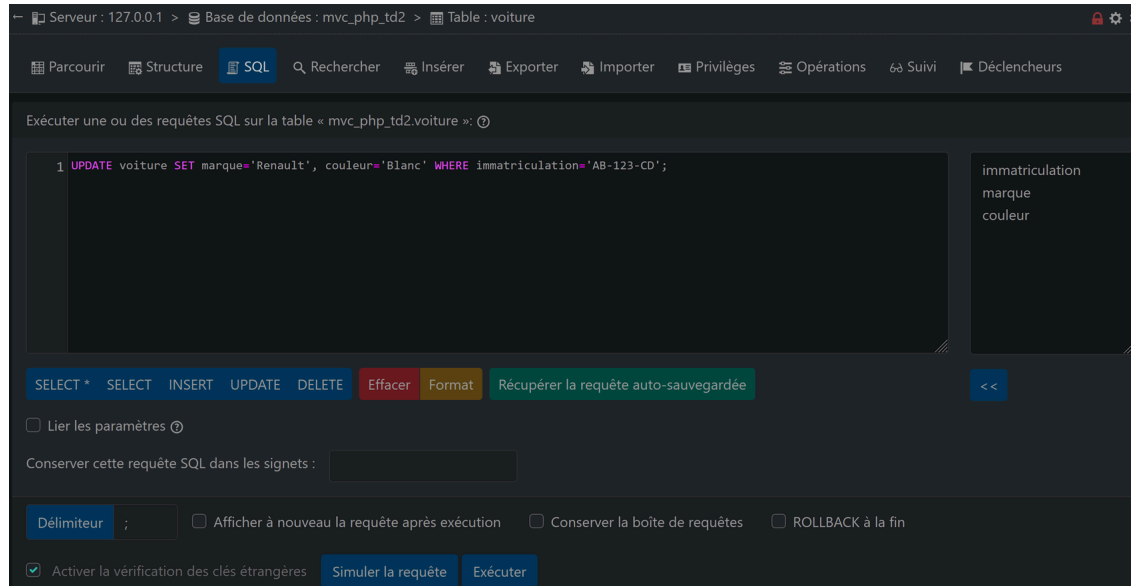
Marque : Renault

Couleur : Bleu

Envoyer

Exercice 5

On teste d'abord la requête **SQL** dans **phpMyAdmin** avant de faire la fonction :



Fonction “**update(\$data)**” dans le modèle, **\$data** est un tableau associatif qui est directement donné à **execute()** :

```
public static function update($data) {
    $sql = "UPDATE voiture SET marque=:marque, couleur=:couleur WHERE immatriculation=:immatriculation";
    $req_prep = Model::$pdo->prepare($sql);
    $req_prep->execute($data);
}
```

Vue “updated.php” et action **updated** :

```
<p>La voiture d'immatriculation <?php echo htmlspecialchars($immat); ?> a bien été mise à jour !</p>
<?php require File::build_path(array("view", "voiture", "list.php")); ?>
```

```
public static function updated() {
    $data = array(
        "immatriculation" => $_GET['immatriculation'],
        "marque" => $_GET['marque'],
        "couleur" => $_GET['couleur'],
    );
    ModelVoiture::update($data);

    $immat = $_GET['immatriculation'];
    $tab_v = ModelVoiture::getAllVoitures();
    $controller = 'voiture';
    $view = 'updated';
    $pagetitle = 'Voiture modifiée';
    require File::build_path(array("view", "view.php"));
}
```

Test : on change la couleur de la voiture “**AB-123-CD**” :

http://localhost/public_html/PHP/TD6/index.php?action=update&immat=AB-123-CD

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Modifier la voiture :

Immatriculation :

Marque :

Couleur :

 Noir

Voitures | Utilisateurs | Trajets

La voiture d'immatriculation AB-123-CD a bien été mise à jour !

Liste des voitures :

Voiture d'immatriculation [&immat=h](#).

Voiture d'immatriculation [00-000-00](#).

Voiture d'immatriculation [AA-000-AA](#).

Voiture d'immatriculation [AB-123-CD](#).

Voiture d'immatriculation [CI-789-TR](#).

Voiture d'immatriculation [FI-456-AT](#).

Voiture d'immatriculation [XX-999-YY](#).

[Ajouter une voiture](#)

Voitures | Utilisateurs | Trajets

Voiture AB-123-CD de la marque Renault (couleur Noir)

[Modifier cette voiture](#)

[Supprimer cette voiture](#)

[Retour à la liste](#)

Exercice 6

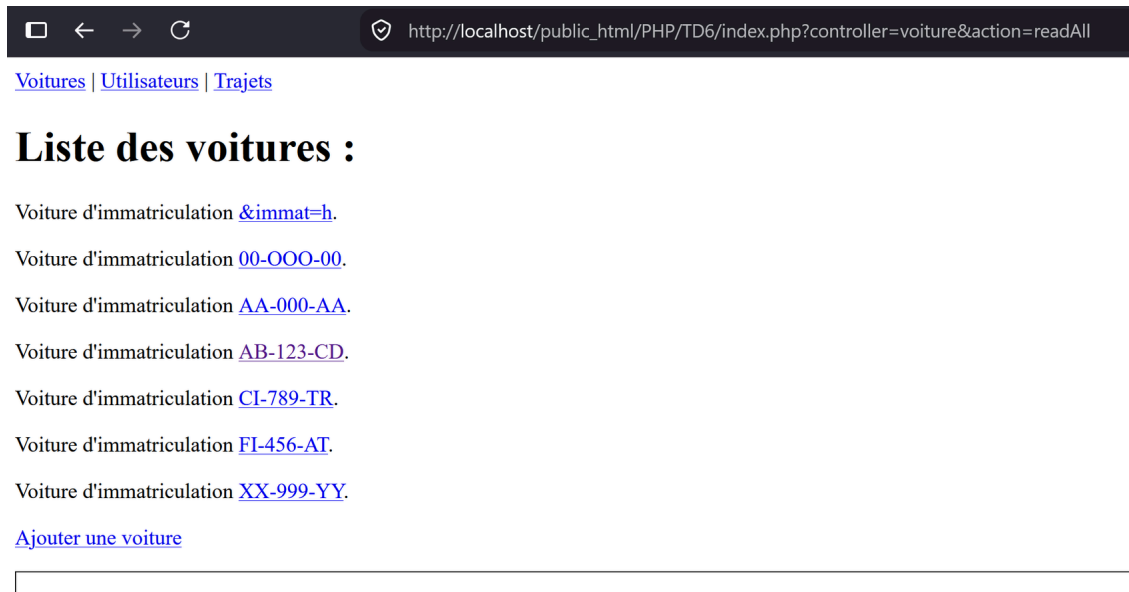
On récupère le contrôleur dans l'URL (**voiture** par défaut), on construit le nom de la classe avec **ucfirst()** et on teste si elle existe avec **class_exists()** :

```
if (isset($_GET['controller'])) {
    $controller = $_GET['controller'];
} else {
    $controller = 'voiture';
}

$controller_class = 'Controller' . ucfirst($controller);

if (class_exists($controller_class)) {
    if (in_array($action, get_class_methods($controller_class))) {
        $controller_class::$action();
    } else {
        ControllerVoiture::error();
    }
} else {
    ControllerVoiture::error();
}
?>
```

Les anciennes pages marchent toujours, avec ou sans le paramètre **controller** :



Exercice 7

Création de “**controller/ControllerUtilisateur.php**” avec seulement l’action **readAll** :

```
<?php
require_once File::build_path(array("model", "ModelUtilisateur.php")); // changement du modèle

class ControllerUtilisateur {

    public static function readAll() {
        $tab_u = ModelUtilisateur::getAllUtilisateurs(); //appel au modèle pour gerer la BD
        $controller = 'utilisateur';
        $view = 'list';
        $pagetitle = 'Liste des utilisateurs';
        require File::build_path(array("view", "view.php"));
    }
}
```

Question : « Quelles différences notez-vous entre le code de ControllerUtilisateur.php et celui de ControllerVoiture.php ? »

Réponse : Le code est quasiment le même, seuls changent le modèle chargé (**ModelUtilisateur**), la fonction appelée (**getAllUtilisateurs()**), le nom de la variable (**\$tab_u**), le **\$controller** et le titre de la page.

On charge ce contrôleur dans le routeur :

```
<?php
require_once File::build_path(array("controller", "ControllerVoiture.php"));
require_once File::build_path(array("controller", "ControllerUtilisateur.php"));
```

Création du modèle “**model/ModelUtilisateur.php**” (getters, setters, constructeur et **getAllUtilisateurs()**) :

```
public function __construct($l = NULL, $n = NULL, $p = NULL) {
    if (!is_null($l) && !is_null($n) && !is_null($p)) {
        $this->login = $l;
        $this->nom = $n;
        $this->prenom = $p;
    }
}

public static function getAllUtilisateurs() {
    $sql = "SELECT * FROM utilisateur";
    $rep = Model::$pdo->query($sql);
    $rep->setFetchMode(PDO::FETCH_CLASS, 'ModelUtilisateur');
    return $rep->fetchAll();
}
?>
```

Question : « Quelles différences notez-vous entre le code de ModelUtilisateur.php et celui de ModelVoiture.php ? »

Réponse : Là aussi c’est presque pareil : seuls le nom de la table (**utilisateur**), le nom de la classe donnée à **setFetchMode()** et les attributs changent. Il y a donc beaucoup de code en double.

Vue “**view/utilisateur/list.php**” :

```
<h1>Liste des utilisateurs :</h1>
<?php
foreach ($tab_u as $u) {
    $uLogin = htmlspecialchars($u->getLogin());
    echo '<p> Utilisateur ' . $uLogin . '</p>';
}
?>
```

Test avec le lien “**Utilisateurs**” du menu :



Exercice 8

On déplace “**getAllVoitures()**” dans “**Model.php**” en la renommant “**selectAll()**”.
Le nom de la table et de la classe sont récupérés avec **static::\$object** :

```
public static function selectAll() {
    $table_name = static::$object;
    $class_name = 'Model' . ucfirst(static::$object);

    $sql = "SELECT * FROM $table_name";
    $rep = Model::$pdo->query($sql);
    $rep->setFetchMode(PDO::FETCH_CLASS, $class_name);
    return $rep->fetchAll();
}
```

ModelVoiture hérite maintenant de **Model** et contient une variable **\$object** (protected static) :

```
<?php
require_once File::build_path(array("model", "Model.php"));

class ModelVoiture extends Model {

    protected static $object = 'voiture';

    private $marque;
    private $couleur;
    private $immatriculation;
```

Pareil pour **ModelUtilisateur** :

```
<?php
require_once File::build_path(array("model", "Model.php"))

class ModelUtilisateur extends Model {

    protected static $object = 'utilisateur';

    private $login;
    private $nom;
    private $prenom;
```

On appelle maintenant **ModelVoiture::selectAll()** et **ModelUtilisateur::selectAll()** dans les contrôleurs :

```
public static function readAll() {
    $tab_v = ModelVoiture::selectAll(); //appel au modèle pour gerer la BD
    $controller = 'voiture';
    $view = 'list';
    $pagetitle = 'Liste des voitures';
    require File::build_path(array("view", "view.php"));
}
```

Le site marche toujours :



Voitures | Utilisateurs | Trajets

Liste des utilisateurs :

Utilisateur jdupont.

Utilisateur mbardet.

--

Exercice 9

Même principe pour “**getVoitureByImmat()**” qui devient “**select(\$primary_value)**” dans “**Model.php**”, avec la clé primaire stockée dans **static::\$primary** :

```
public static function select($primary_value) {
    $table_name = static::$object;
    $class_name = 'Model' . ucfirst(static::$object);
    $primary_key = static::$primary;

    $sql = "SELECT * FROM $table_name WHERE $primary_key=:nom_tag";
    // Préparation de la requête
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "nom_tag" => $primary_value,
    );
    $req_prep->execute($values);

    $req_prep->setFetchMode(PDO::FETCH_CLASS, $class_name);
    $tab = $req_prep->fetchAll();
    // Attention, si il n'y a pas de résultats, on renvoie false
    if (empty($tab))
        return false;
    return $tab[0];
}
```

```
class ModelVoiture extends Model {

    protected static $object = 'voiture';
    protected static $primary = 'immatriculation';
}
```

(Dans **ModelUtilisateur** la clé primaire est **login**). L'action **read()** appelle maintenant **ModelVoiture::select(\$immat)** :

```
public static function read() {
    $immat = $_GET['immat'];
    $v = ModelVoiture::select($immat);
    if ($v === false) {
        self::error();
    } else {
        $controller = 'voiture';
        $view = 'detail';
        $pagetitle = 'Détail de la voiture';
        require File::build_path(array("view", "view.php"));
    }
}
```



Exercice 10

On ajoute un attribut **protected static \$object** dans les contrôleurs, et la vue générique utilise **static::\$object** à la place de la variable **\$controller** :

```
class ControllerVoiture {  
  
    protected static $object = 'voiture';  
  
    public static function readAll() {  
        $tab_v = ModelVoiture::selectAll();    //appel au modèle pour gerer la BD  
        $view = 'list';  
        $pagetitle = 'Liste des voitures';  
        require File::build_path(array("view", "view.php"));  
    }  
}
```

```
<?php  
$filepath = File::build_path(array("view", static::$object, "$view.php"));  
require $filepath;  
?>
```

Ça marche car la vue est incluse depuis une méthode du contrôleur, donc **static::** désigne la classe du contrôleur appelé.

Exercice 11

Action `read()` de `ControllerUtilisateur` :

```
public static function read() {
    $login = $_GET['login'];
    $u = ModelUtilisateur::select($login);
    if ($u === false) {
        self::error();
    } else {
        $view = 'detail';
        $pagetitle = 'Détail de l\'utilisateur';
        require File::build_path(array("view", "view.php"));
    }
}
```

Vue `"detail.php"` et liens dans `"list.php"` :

```
<?php
$uLogin = htmlspecialchars($u->getLogin());
$uNom = htmlspecialchars($u->getNom());
$uPrenom = htmlspecialchars($u->getPrenom());
$uLoginURL = rawurlencode($u->getLogin());

echo "<p>Utilisateur " . $uLogin . " : " . $uPrenom . " " . $uNom . "</p>";
?>
<a href="index.php?controller=utilisateur&action=readAll">Retour à la liste</a>
```

```
<?php
foreach ($tab_u as $u) {
    $uLogin = htmlspecialchars($u->getLogin());
    $uLoginURL = rawurlencode($u->getLogin());
    echo '<p> Utilisateur <a href="index.php?controller=utilisateur&action=read&login=' . $uLoginURL . '>' . $uLogin . '
?>
```

Voitures | Utilisateurs | Trajets

Voitures | Utilisateurs | Trajets

Liste des utilisateurs :

Utilisateur [jdupont](#).

Utilisateur [mbardet](#).

Voitures | Utilisateurs | Trajets

Voitures | Utilisateurs | Trajets

Utilisateur [jdupont](#) : Jean Dupont

[Retour à la liste](#)

Exercice 12

Fonction générique “`delete($primary_value)`” dans “**Model.php**” :

```
public static function delete($primary_value) {
    $table_name = static::$object;
    $primary_key = static::$primary;

    $sql = "DELETE FROM $table_name WHERE $primary_key=:nom_tag";
    $req_prep = Model::$pdo->prepare($sql);
    $values = array(
        "nom_tag" => $primary_value,
    );
    $req_prep->execute($values);
}
```

Action **delete** de **ControllerUtilisateur** (**ControllerVoiture** utilise aussi **ModelVoiture::delete()** maintenant) et vue “**deleted.php**” :

```
public static function delete() {
    $login = $_GET['login'];
    ModelUtilisateur::delete($login);
    $tab_u = ModelUtilisateur::selectAll();
    $view = 'deleted';
    $pagetitle = 'Utilisateur supprimé';
    require File::build_path(array("view", "view.php"));
}
```

```
<p>L'utilisateur <?php echo htmlspecialchars($login); ?> a bien été supprimé !</p>
<?php require File::build_path(array("view", "utilisateur", "list.php")); ?>
```

Liens pour supprimer dans “**list.php**” :

```
<?php
foreach ($tab_u as $u) {
    $uLogin = htmlspecialchars($u->getLogin());
    $uLoginURL = rawurlencode($u->getLogin());
    echo '<p> Utilisateur <a href="index.php?controller=utilisateur&action=read&login=' . $uLoginURL . '"> . $uLogin . '</a>'
        . ' <a href="index.php?controller=utilisateur&action=delete&login=' . $uLoginURL . '">(supprimer)</a></p>';
}
?>
```

Pour tester, j'ai ajouté un utilisateur "ttest" dans la **BDD** :



Exercice 13

On supprime la vue "create.php" : l'action **create** affiche aussi "update.php" avec des champs vides. Deux variables permettent d'adapter la vue : **\$etat_immat** (**required** ou **readonly**) et **\$action_form** :

```
public static function create() {  
    $immat = "";  
    $marque = "";  
    $couleur = "";  
    $etat_immat = "required";  
    $action_form = "created";  
    $view = 'update';  
    $pagetitle = 'Créer une voiture';  
    require File::build_path(array("view", "view.php"));  
}
```

```

public static function update() {
    $v = ModelVoiture::select($_GET['immat']);
    if ($v === false) {
        self::error();
    } else {
        $immat = $v->getImmatriculation();
        $marque = $v->getMarque();
        $couleur = $v->getCouleur();
        $etat_immat = "readonly";
        $action_form = "updated";
        $view = 'update';
        $pagetitle = 'Modifier une voiture';
        require File::build_path(array("view", "view.php"));
    }
}

```

La vue fusionnée, avec en plus le champ caché **controller** (récupéré avec `static::$object`) :

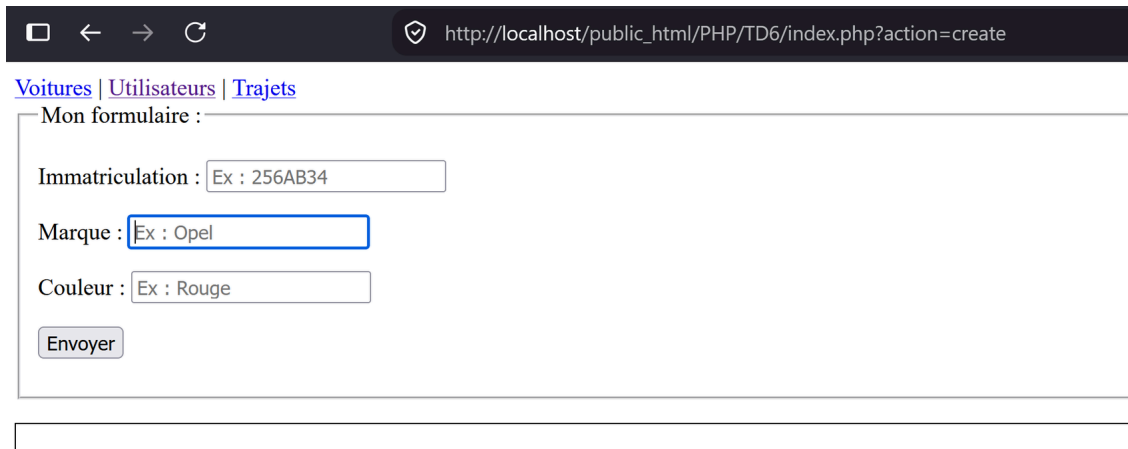
```

<?php
$vImmat = htmlspecialchars($immat);
$vMarque = htmlspecialchars($marque);
$vCouleur = htmlspecialchars($couleur);
?>
<form method="get" action="index.php">
    <fieldset>
        <legend>Mon formulaire :</legend>
        <input type="hidden" name="action" value="<?php echo $action_form; ?>" />
        <input type="hidden" name="controller" value="<?php echo static::$object; ?>" />
        <p>
            <label for="immat_id">Immatriculation</label> :
            <input type="text" placeholder="Ex : 256AB34" name="immatriculation" id="immat_id" value="<?php echo $vImmat; ?>" />
        </p>
        <p>
            <label for="marque_id">Marque</label> :
            <input type="text" placeholder="Ex : Opel" name="marque" id="marque_id" value="<?php echo $vMarque; ?>" required />
        </p>
        <p>
            <label for="couleur_id">Couleur</label> :
            <input type="text" placeholder="Ex : Rouge" name="couleur" id="couleur_id" value="<?php echo $vCouleur; ?>" required />
        </p>
        <input type="submit" value="Envoyer" />
    </fieldset>
</form>

```

(Le sujet parle d'une action save pour la création, mais notre action s'appelle created depuis le TD4, on garde donc created).

Création (immatriculation obligatoire) :



Voitures | Utilisateurs | Trajets

Mon formulaire :

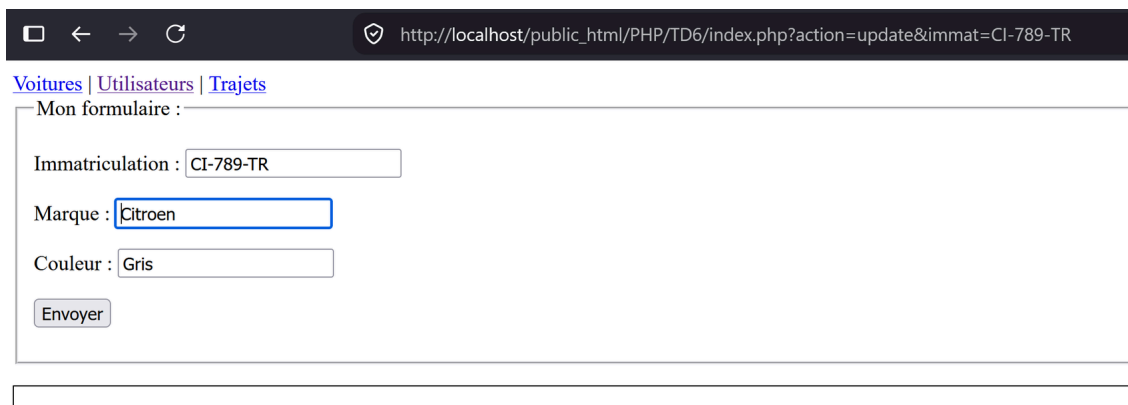
Immatriculation : Ex : 256AB34

Marque : Ex : Opel

Couleur : Ex : Rouge

Envoyer

Mise à jour (immatriculation en lecture seule) :



Voitures | Utilisateurs | Trajets

Mon formulaire :

Immatriculation : CI-789-TR

Marque : Citroen

Couleur : Gris

Envoyer

Exercice 14

Même chose pour les utilisateurs, actions **create** et **update** :

```
public static function create() {  
    $login = "";  
    $nom = "";  
    $prenom = "";  
    $etat_login = "required";  
    $action_form = "created";  
    $view = 'update';  
    $pagetitle = 'Créer un utilisateur';  
    require File::build_path(array("view", "view.php"))  
}
```

```
public static function update() {  
    $u = ModelUtilisateur::select($_GET['login']);  
    if ($u === false) {  
        self::error();  
    } else {  
        $login = $u->getLogin();  
        $nom = $u->getNom();  
        $prenom = $u->getPrenom();  
        $etat_login = "readonly";  
        $action_form = "updated";  
        $view = 'update';  
        $pagetitle = 'Modifier un utilisateur';  
        require File::build_path(array("view", "view.php"));  
    }  
}
```

Vue “view/utilisateur/update.php” :

```
<?php
$uLogin = htmlspecialchars($login);
$uNom = htmlspecialchars($nom);
$uPrenom = htmlspecialchars($prenom);
?>
<form method="get" action="index.php">
  <fieldset>
    <legend>Mon formulaire :</legend>
    <input type="hidden" name="action" value="<?php echo $action_form; ?>" />
    <input type="hidden" name="controller" value="<?php echo static::$object; ?>" />
    <p>
      <label for="login_id">login</label> :
      <input type="text" placeholder="Ex : jdupont" name="login" id="login_id" value="<?php echo $uLogin; ?>" <?php echo $etat
    </p>
    <p>
      <label for="nom_id">Nom</label> :
      <input type="text" placeholder="Ex : Dupont" name="nom" id="nom_id" value="<?php echo $uNom; ?>" required autofocus/>
    </p>
    <p>
      <label for="prenom_id">Prénom</label> :
      <input type="text" placeholder="Ex : Jean" name="prenom" id="prenom_id" value="<?php echo $uPrenom; ?>" required/>
    </p>
    <input type="submit" value="Envoyer" />
  </fieldset>
</form>
```

Et le lien “Modifier cet utilisateur” dans “detail.php” :

The screenshot shows a web browser window with the URL `http://localhost/public_html/PHP/TD6/index.php?controller=utilisateur&action=read&login=mbardet`. The page displays navigation links: [Voitures](#), [Utilisateurs](#), and [Trajets](#). Below these, it shows the user details: "Utilisateur mbardet : Marie Bardet". A link [Modifier cet utilisateur](#) is highlighted with a red box. Below this link is a "Retour à la liste" link. The main content area is empty.

The screenshot shows a web browser window with the URL `http://localhost/public_html/PHP/TD6/index.php?controller=utilisateur&action=update&login=mbardet`. The page displays navigation links: [Voitures](#), [Utilisateurs](#), and [Trajets](#). Below these, it shows the title "Mon formulaire :". The form contains three text input fields: "Login : mbardet", "Nom : Bardet", and "Prénom : Marie". Below these fields is an "Envoyer" button. The main content area is empty.

Exercice 15

On déplace “**update()**” dans “**Model.php**”. La partie **SET** de la requête est générée à partir des clés du tableau **\$data** avec un **foreach**, et **rtrim()** enlève la virgule en trop :

```
public static function update($data) {
    $table_name = static::$object;
    $primary_key = static::$primary;

    $set = "";
    foreach ($data as $cle => $valeur) {
        $set = $set . "$cle=: $cle,";
    }
    $set = rtrim($set, ",");

    $sql = "UPDATE $table_name SET $set WHERE $primary_key=: $primary_key";
    $req_prep = Model::$pdo->prepare($sql);
    $req_prep->execute($data);
}
```

Action **updated** des utilisateurs :

```
public static function updated() {
    $data = array(
        "login" => $_GET['login'],
        "nom" => $_GET['nom'],
        "prenom" => $_GET['prenom'],
    );
    ModelUtilisateur::update($data);

    $login = $_GET['login'];
    $tab_u = ModelUtilisateur::selectAll();
    $view = 'updated';
    $pagetitle = 'Utilisateur modifié';
    require File::build_path(array("view", "view.php"));
}
```

Test : on modifie le prénom de “mbardet” :

Voitures | Utilisateurs | Trajets

Mon formulaire :

Login :

Nom :

Prénom :

Voitures | Utilisateurs | Trajets

L'utilisateur mbardet a bien été mis à jour !

Liste des utilisateurs :

Utilisateur [jdupont](#) ([supprimer](#))

Utilisateur [mbardet](#) ([supprimer](#))

[Ajouter un utilisateur](#)

Voitures | Utilisateurs | Trajets

Utilisateur mbardet : Marion Bardet

[Modifier cet utilisateur](#)

[Retour à la liste](#)

Exercice 16

Même chose avec **“save()”** : on génère la liste des champs et des tags à partir des clés de **\$data**. J’ai ajouté un **try/catch** pour renvoyer **false** si la clé primaire existe déjà :

```
public static function save($data) {
    $table_name = static::$object;

    $champs = "";
    $tags = "";
    foreach ($data as $cle => $valeur) {
        $champs = $champs . "$cle,";
        $tags = $tags . ":$cle,";
    }
    $champs = rtrim($champs, ",");
    $tags = rtrim($tags, ",");

    $sql = "INSERT INTO $table_name ($champs) VALUES ($tags)";
    $req_prep = Model::$pdo->prepare($sql);
    try {
        $req_prep->execute($data);
    } catch (PDOException $e) {
        return false;
    }
    return true;
}
```

Les actions **created** utilisent maintenant **save(\$data)** :

```
public static function created() {
    $data = array(
        "immatriculation" => $_GET['immatriculation'],
        "marque" => $_GET['marque'],
        "couleur" => $_GET['couleur'],
    );
    if (ModelVoiture::save($data) === false) {
        self::error();
    } else {
        $tab_v = ModelVoiture::selectAll();
        $view = 'created';
        $pagetitle = 'Voiture créée';
        require File::build_path(array("view", "view.php"));
    }
}
```

```

public static function created() {
    $data = array(
        "login" => $_GET['login'],
        "nom" => $_GET['nom'],
        "prenom" => $_GET['prenom'],
    );
    if (ModelUtilisateur::save($data) === false) {
        self::error();
    } else {
        $tab_u = ModelUtilisateur::selectAll();
        $view = 'created';
        $pagetitle = 'Utilisateur créé';
        require File::build_path(array("view", "view.php"));
    }
}

```

Test en créant un utilisateur :

Voitures | Utilisateurs | Trajets

Mon formulaire :

Login :

Nom :

Prénom :

Voitures | Utilisateurs | Trajets

L'utilisateur a bien été créé !

Liste des utilisateurs :

Utilisateur [jdupont](#) ([supprimer](#))

Utilisateur [mbardet](#) ([supprimer](#))

Utilisateur [pmartin](#) ([supprimer](#))

[Ajouter un utilisateur](#)

Contrôleur trajet

Grâce au modèle générique, **ModelTrajet** ne contient plus que ses attributs, **\$object**, **\$primary** et la fonction “**findPassagers()**” du TD3 :

```
<?php
require_once File::build_path(array("model", "Model.php"));

class ModelTrajet extends Model {

    protected static $object = 'trajet';
    protected static $primary = 'id';

    private $id;
    private $depart;
    private $arrivee;
    private $date;
    private $nbplaces;
    private $prix;
    private $conducteur_login;
```

ControllerTrajet reprend les mêmes actions que les autres contrôleurs (**read**, **delete**, **create**, **update**, **created** et **updated**), par exemple :

```
public static function read() {
    $t = ModelTrajet::select($_GET['id']);
    if ($t === false) {
        self::error();
    } else {
        $tab_p = ModelTrajet::findPassagers($t->getId());
        $view = 'detail';
        $pagetitle = 'Détail du trajet';
        require File::build_path(array("view", "view.php"));
    }
}
```

```

public static function created() {
    $data = array(
        "depart" => $_GET['depart'],
        "arrivee" => $_GET['arrivee'],
        "date" => $_GET['date'],
        "nbplaces" => $_GET['nbplaces'],
        "prix" => $_GET['prix'],
        "conducteur_login" => $_GET['conducteur_login'],
    );
    if (ModelTrajet::save($data) === false) {
        self::error();
    } else {
        $tab_t = ModelTrajet::selectAll();
        $view = 'created';
        $pagetitle = 'Trajet créé';
        require File::build_path(array("view", "view.php"));
    }
}
}

```

Dans le formulaire, le conducteur est choisi dans une liste des utilisateurs :

```

<label for="conducteur_id">Conducteur</label> :
<select name="conducteur_login" id="conducteur_id">
<?php
foreach ($tab_u as $u) {
    $selected = ($u->getLogin() == $conducteur_login) ? " selected" : "";
    echo '    <option value="' . htmlspecialchars($u->getLogin()) . '" ' . $selected . '>' . htmlspecialchars($u->getLog
}
?>
</select>

```

Liste des trajets :

⏏ ⬅ ➡ ↺ http://localhost/public_html/PHP/TD6/index.php?controller=trajet

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Liste des trajets :

[Trajet n°1](#) : Gap → Marseille ([supprimer](#))

[Trajet n°3](#) : Briançon → Gap ([supprimer](#))

[Ajouter un trajet](#)

Création d'un trajet :

Voitures | Utilisateurs | Trajets

Mon formulaire :

Départ :

Arrivée :

Date :

Nombre de places :

Prix : €

Conducteur :

Voitures | Utilisateurs | Trajets

Le trajet a bien été créé !

Liste des trajets :

[Trajet n°1](#) : Gap → Marseille ([supprimer](#))

[Trajet n°3](#) : Briançon → Gap ([supprimer](#))

[Trajet n°5](#) : Gap → Grenoble ([supprimer](#))

[Ajouter un trajet](#)

Pour la liste des passagers (table de jointure **passager** du **TD3**), j'ai inscrit deux passagers au trajet 1 dans la **BDD**. La vue de détail d'un trajet affiche ses passagers :

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Trajet n°1 : Gap → Marseille

Le 2026-09-15, 3 place(s), 20 €

Conducteur : [jdupont](#)

Passagers :

Marion Bardet (mbardet)

Paul Martin (pmartin)

[Modifier ce trajet](#)

[Supprimer ce trajet](#)

[Retour à la liste](#)

Et inversement, le détail d'un utilisateur affiche les trajets où il est passager :

[Voitures](#) | [Utilisateurs](#) | [Trajets](#)

Utilisateur pmartin : Paul Martin

Trajets en tant que passager :

[Trajet n°1](#) : Gap → Marseille

[Modifier cet utilisateur](#)

[Retour à la liste](#)

Modification d'un trajet :

Voitures | Utilisateurs | Trajets

Mon formulaire :

Numéro : 3

Départ : Briançon

Arrivée : Gap

Date : 18 / 09 / 2026

Nombre de places : 4

Prix : 10 €

Conducteur : mbardet

Envoyer

Conclusion

Ce **TD** nous a permis de finir le **CRUD** des voitures puis de le généraliser aux utilisateurs et aux trajets grâce à un modèle générique (**selectAll**, **select**, **delete**, **update**, **save**) et à la variable **static::\$object**. Ajouter un nouvel objet demande maintenant beaucoup moins de code.